

Dataplex MCP Server

Technical Documentation

Version: 0.1.0

Status: Alpha

License: Apache-2.0

Generated: 2026-04-09

Table of Contents

1. Overview
2. Getting Started
3. Configuration
4. Tool Reference
5. Architecture
6. Development
7. Troubleshooting

Dataplex MCP Server

Version: 0.1.0 **Status:** Alpha **License:** Apache-2.0

The **Dataplex MCP Server** is a [Model Context Protocol](#) server that exposes [Google Cloud Dataplex](#) resources and operations as tools that any MCP-capable Large Language Model (LLM) client can call. It lets an agent explore a Dataplex estate in natural language — listing lakes, inspecting schemas, running data quality scans, triggering tasks — without hand-writing gcloud or REST calls.

What is MCP?

The Model Context Protocol is an open standard that lets LLM applications connect to external tools and data sources in a uniform way. An **MCP server** exposes a set of named tools, each with a JSON Schema describing its arguments. An **MCP host** (Claude Desktop, Claude Code, Cursor, or any compatible client) discovers those tools and surfaces them to the underlying model, which then chooses when to call them.

What this server exposes

Eighteen tools across five functional areas:

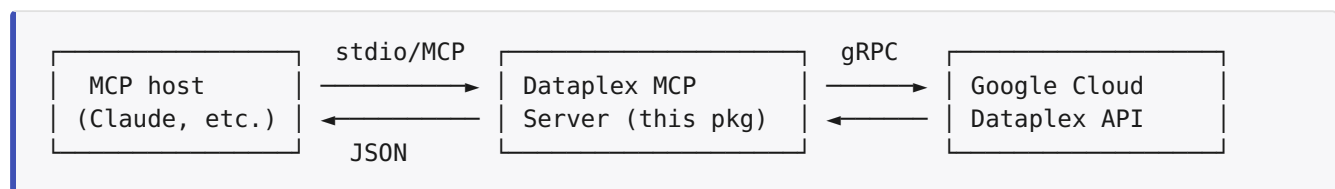
Area	Tools
Lakes / zones / assets	<code>list_lakes</code> , <code>get_lake</code> , <code>list_zones</code> , <code>get_zone</code> , <code>list_assets</code> , <code>get_asset</code>
Metadata	<code>list_entities</code> , <code>get_entity</code> , <code>list_partitions</code>
Tasks	<code>list_tasks</code> , <code>get_task</code> , <code>run_task</code> , <code>list_jobs</code>
Data scans (DQ & profile)	<code>list_data_scans</code> , <code>get_data_scan</code> , <code>run_data_scan</code> , <code>list_data_scan_jobs</code> , <code>get_data_scan_job</code>

All tools return JSON-serialized results that downstream models can parse and reason over.

Who should use it

- **Data platform teams** who want an LLM assistant that can answer questions like *"Which zones in our `retail` lake have failing data quality scans?"* or *"Trigger the `hourly-ingest` task and tell me when it's done."*
- **Analytics engineers** who want schema lookup and partition introspection from within their chat client.
- **Site reliability engineers** building runbooks that drive Dataplex operations from an AI copilot.

Architecture at a glance



The server is a thin adapter: the MCP layer handles protocol framing and tool registration, a facade class wraps the `google-cloud-dataplex` SDK, and every tool result is converted to a plain JSON object before being returned to the host.

See Architecture for details.

Documentation map

1. Getting Started — install, authenticate, and make your first tool call.
2. Configuration — environment variables, defaults, authentication methods, IAM roles.
3. Tool Reference — every tool, its arguments, and its response shape.
4. Architecture — module layout, request flow, design decisions.
5. Development — contributing, testing, extending with new tools.
6. Troubleshooting — common errors and fixes.

Support

- **Issues / feature requests:** file a ticket in the parent CDAP repository.
- **Security disclosures:** follow the policy in the root `SECURITY.md` or `CONTRIBUTING.rst` of the repository.

Getting Started

This guide walks you through installing the Dataplex MCP Server, authenticating to Google Cloud, connecting it to an MCP host, and making your first tool call.

Prerequisites

- **Python 3.10+** — check with `python --version`.
- **A Google Cloud project** with the Dataplex API enabled. Enable it with:

```
gcloud services enable dataplex.googleapis.com
```

- **IAM permissions.** At minimum `roles/dataplex.viewer` on the project. Add `roles/dataplex.editor` if you want to use `run_task` or `run_data_scan`.
- **An MCP host** — Claude Desktop, Claude Code, or any other client that can launch a stdio MCP server.

Install

From source (recommended while in alpha)

```
git clone https://github.com/cpydimuk/cdap.git
cd cdap/dataplex-mcp-server
pip install -e .
```

The editable install exposes a `dataplex-mcp-server` console script. Verify it:

```
which dataplex-mcp-server
dataplex-mcp-server --help 2>&1 || true    # the server speaks MCP; no CLI help
```

Isolated install with a virtual environment

Recommended if you don't want to pollute your system Python:

```
python -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\activate
pip install -e .
```

Any MCP host launching the server needs to point to this virtualenv's `dataplex-mcp-server` binary (or invoke `python -m dataplex_mcp_server` using the virtualenv's Python).

Authenticate

The server uses **Application Default Credentials (ADC)**. Choose the method that matches your environment:

Environment	Recommended method
Local developer workstation	<code>gcloud auth application-default login</code>
CI / build agent / container	<code>GOOGLE_APPLICATION_CREDENTIALS</code> pointing to a service-account key file
GCE, GKE, Cloud Run, Cloud Functions	Metadata server — no setup required
Workload Identity Federation	Configured credentials file via ADC

For a user login:

```
gcloud auth application-default login
```

For a service-account key:

```
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/key.json
```

Security note: treat service-account key files like passwords. Prefer Workload Identity Federation or short-lived credentials in production.

Set default project and region

Two environment variables let every tool call omit `project_id` and `location`:

```
export DATAPLEX_PROJECT_ID=my-gcp-project
export DATAPLEX_LOCATION=us-central1
```

`DATAPLEX_PROJECT_ID` falls back to `GOOGLE_CLOUD_PROJECT` if unset. You can always override per call by passing the arguments explicitly.

Smoke test

Before wiring the server into an MCP host, confirm it can start and talk to Dataplex. The quickest path is to call a handler directly:

```
# smoke_test.py
from dataplex_mcp_server.dataplex_client import DataplexClient, DataplexConfig
from dataplex_mcp_server.server import _build_tool_registry

client = DataplexClient(DataplexConfig(
    project_id="my-gcp-project",
    location="us-central1",
))
_, handlers = _build_tool_registry(client)

print(handlers["list_lakes"]())
```

```
python smoke_test.py
```

A successful run prints a (possibly empty) list of lake dictionaries. Errors point at authentication, permissions, or the wrong region; see [Troubleshooting](#).

Connect to an MCP host

Claude Desktop

Edit your Claude Desktop config file:

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`
- **Linux:** `~/.config/Claude/claude_desktop_config.json`

Add a `dataplex` entry under `mcpServers`:

```
{
  "mcpServers": {
    "dataplex": {
      "command": "dataplex-mcp-server",
      "env": {
        "DATAPLEX_PROJECT_ID": "my-gcp-project",
        "DATAPLEX_LOCATION": "us-central1",
        "GOOGLE_APPLICATION_CREDENTIALS": "/Users/me/.config/gcloud/sa.json"
      }
    }
  }
}
```

Restart Claude Desktop. The Dataplex tools appear in the tools panel.

Claude Code (CLI)

```
claude mcp add dataplex -- dataplex-mcp-server
```

Then export the environment variables in the shell that launches `claude`, or edit the generated MCP config entry to set them inline.

Other MCP hosts

Any host that can launch a stdio MCP server works. Point the host at the `dataplex-mcp-server` executable (or `python -m dataplex_mcp_server`) and pass `DATAPLEX_PROJECT_ID`, `DATAPLEX_LOCATION`, and `GOOGLE_APPLICATION_CREDENTIALS` via its environment configuration.

Your first prompt

With the server connected, ask the host something like:

"List all Dataplex lakes in the project and summarize them."

The model will call `list_lakes`, receive a JSON array, and format it into a human-readable summary. Follow-up prompts can drill down:

"Show me the zones in the `retail` lake, and for each list the assets."

This chains `list_zones` → `list_assets` transparently.

Next steps

- Read the [Tool Reference](#) for the full API surface.
- Learn how [Configuration defaults and overrides](#) interact.
- Extend the server with new tools in the [Development guide](#).

Configuration

This page documents every knob the Dataplex MCP Server exposes: environment variables, authentication, IAM roles, transport, and per-call overrides.

Environment variables

Variable	Purpose	Default	Required?
<code>DATAPLEX_PROJECT_ID</code>	Default Google Cloud project ID for tool calls.	—	One of this or <code>GOOGLE_CLOUD_PROJECT</code> must be set, unless every tool call passes <code>project_id</code> explicitly.
<code>GOOGLE_CLOUD_PROJECT</code>	Fallback default project ID.	—	Only used if <code>DATAPLEX_PROJECT_ID</code> is not set.
<code>DATAPLEX_LOCATION</code>	Default Dataplex region (e.g. <code>us-central1</code>).	—	Must be set unless every tool call passes <code>location</code> explicitly.
<code>GOOGLE_APPLICATION_CREDENTIALS</code>	Path to a service-account key JSON file.	—	Only if ADC cannot be resolved another way.

Precedence for resolving project and location at a tool call site:

1. Explicit `project_id` / `location` arguments on the tool call.
2. `DataplexConfig` object passed into `DataplexClient`.
3. `DATAPLEX_PROJECT_ID` env var (or `GOOGLE_CLOUD_PROJECT` fallback).
4. `DATAPLEX_LOCATION` env var.

If neither layer resolves a value, the server raises a `ValueError` with a clear message (`project_id is required...` or `location is required...`).

Authentication

The server uses **Application Default Credentials (ADC)** — it never handles API keys directly. ADC is resolved in the following order by the `google-auth` library:

- 1. `GOOGLE_APPLICATION_CREDENTIALS` environment variable pointing at a service-account key file.
- 2. User credentials from `gcloud auth application-default login` (stored under `~/.config/gcloud/application_default_credentials.json`).
- 3. The attached service account on GCE, GKE, Cloud Run, or Cloud Functions (metadata server).
- 4. Workload Identity Federation configured via a credentials file.

Choosing a method

Scenario	Recommendation
Local developer workstation	<code>gcloud auth application-default login</code>
Docker container on a dev laptop	Mount the gcloud ADC file read-only
CI / GitHub Actions	Use Workload Identity Federation — avoid long-lived key files.
Cloud Run / GKE	Attach a service account with the right IAM roles. No key file needed.
Production VM	Use the attached service account via the metadata server.

Service account permissions

Grant the **minimum** role that covers your tool usage:

Tools	Minimum role
Any <code>list_*</code> or <code>get_*</code> tool	<code>roles/dataplex.viewer</code>
<code>run_task</code> , <code>run_data_scan</code>	<code>roles/dataplex.editor</code> (or <code>roles/dataplex.dataScanJobRunner</code> + <code>roles/dataplex.taskUser</code> for finer grain)
Reading metadata via <code>list_entities</code> , <code>get_entity</code> , <code>list_partitions</code>	<code>roles/dataplex.metadataReader</code> (included in <code>viewer</code>)

Grant on the project that hosts the Dataplex resources:

```
gcloud projects add-iam-policy-binding my-gcp-project \
  --member="serviceAccount:dataplex-mcp@my-gcp-project.iam.gserviceaccount.com" \
  --role="roles/dataplex.viewer"
```

Per-call overrides

Every tool accepts `project_id` and `location` arguments. They override the env-var defaults for that single call. This lets a single running server inspect resources across multiple projects or regions without restart.

Example MCP-host prompt that exercises overrides:

```
"Compare the list of lakes in project retail-prod in us-central1 with the lakes in project retail-dev in europe-west1."
```

The model will call `list_lakes` twice, supplying different `project_id` / `location` each time.

Transport

The server communicates over **stdio** — the default for locally-hosted MCP servers. There is no HTTP or WebSocket endpoint. The host launches the server as a subprocess and reads/writes line-delimited MCP frames on the subprocess's stdin/stdout.

Logging is directed to **stderr** so it never collides with the MCP protocol stream.

Logging

The server uses the standard Python `logging` module under the logger name `dataplex_mcp_server`. The default level is `INFO`. To change it, export before launching:

```
export PYTHONLOGLEVEL=DEBUG    # or edit the host env block
```

Or inject configuration programmatically via `logging.basicConfig` if you embed the server in another process.

Client configuration (embedding)

When embedding `DataplexClient` directly (for scripts, tests, or a different protocol layer), use `DataplexConfig`:

```
from dataplex_mcp_server.dataplex_client import DataplexClient, DataplexConfig

client = DataplexClient(DataplexConfig(
    project_id="my-gcp-project",
    location="us-central1",
))
```

Or pull from the environment:

```
client = DataplexClient(DataplexConfig.from_env())
```

`DataplexClient` is lazy: the underlying `DataplexServiceClient`, `MetadataServiceClient`, and `DataScanServiceClient` are only instantiated on first use, so construction is cheap and testable.

Example host configurations

Claude Desktop (macOS)

```
{
  "mcpServers": {
    "dataplex-prod": {
      "command": "/Users/me/.venvs/dataplex/bin/dataplex-mcp-server",
      "env": {
        "DATAPLEX_PROJECT_ID": "retail-prod",
        "DATAPLEX_LOCATION": "us-central1",
        "GOOGLE_APPLICATION_CREDENTIALS": "/Users/me/.config/gcloud/prod-sa.json"
      }
    },
    "dataplex-dev": {
      "command": "/Users/me/.venvs/dataplex/bin/dataplex-mcp-server",
      "env": {
        "DATAPLEX_PROJECT_ID": "retail-dev",
        "DATAPLEX_LOCATION": "europe-west1",
        "GOOGLE_APPLICATION_CREDENTIALS": "/Users/me/.config/gcloud/dev-sa.json"
      }
    }
  }
}
```

Running two instances with different env blocks is the simplest way to keep production and development lanes separate without exposing write permissions on both at once.

Docker

```
FROM python:3.12-slim
WORKDIR /app
COPY dataplex-mcp-server/ /app/
RUN pip install --no-cache-dir .
ENTRYPOINT ["dataplex-mcp-server"]
```

Run with:

```
docker run --rm -i \
  -e DATAPLEX_PROJECT_ID=my-gcp-project \
  -e DATAPLEX_LOCATION=us-central1 \
  -e GOOGLE_APPLICATION_CREDENTIALS=/secrets/sa.json \
  -v /path/to/sa.json:/secrets/sa.json:ro \
  dataplex-mcp-server:latest
```

The `-i` flag is required — stdio MCP servers must have stdin attached.

Tool Reference

Complete reference for every tool exposed by the Dataplex MCP Server.

All tools accept optional `project_id` and `location` arguments that override the environment defaults. Paging-capable tools accept `page_size` (1–1000) and a Dataplex-style `filter` expression. Return values are always JSON and mirror the shape of the underlying Dataplex REST/gRPC resources.

Unless noted otherwise, errors propagate as MCP tool responses with a JSON body of the shape:

```
{ "error": "<ExceptionClassName>", "message": "<human-readable>" }
```

The underlying Google API error codes (`NotFound`, `PermissionDenied`, etc.) are preserved in the exception class name.

Lake tools

`list_lakes`

List Dataplex lakes within a project/location.

Arguments

Name	Type	Required	Description
<code>project_id</code>	string	No	Overrides env default.
<code>location</code>	string	No	Overrides env default.
<code>page_size</code>	integer (1–1000)	No	Server-side paging hint.
<code>filter</code>	string	No	Filter expression (e.g. <code>labels.env=prod</code>).

Returns — JSON array of lake objects. Key fields:

Field	Description
<code>name</code>	Fully qualified resource name (<code>projects/.../locations/.../lakes/...</code>).
<code>display_name</code>	Human-readable name.
<code>state</code>	<code>ACTIVE</code> , <code>CREATING</code> , <code>DELETING</code> , <code>ACTION_REQUIRED</code> .
<code>service_status</code>	Populated status block.
<code>metastore_status</code>	Attached metastore status, if any.
<code>create_time</code> / <code>update_time</code>	RFC 3339 timestamps.

Example call

```
{ "name": "list_lakes", "arguments": { "location": "us-central1" } }
```

`get_lake`

Fetch a single lake.

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No

Returns — A single lake object with the same fields as `list_lakes`.

Zone tools

`list_zones`

List zones inside a lake.

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No
<code>page_size</code>	integer	No
<code>filter</code>	string	No

Returns — JSON array of zone objects. Key fields include `type` (`RAW` or `CURATED`), `resource_spec.location_type` (`SINGLE_REGION` or `MULTI_REGION`), and `discovery_spec`.

`get_zone`

Fetch a single zone.

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>zone_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No

Asset tools

`list_assets`

List assets attached to a zone.

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>zone_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No
<code>page_size</code>	integer	No
<code>filter</code>	string	No

Returns — JSON array of asset objects. Each asset binds a zone to a concrete Cloud Storage bucket or BigQuery dataset via its `resource_spec`.

`get_asset`

Fetch a single asset.

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>zone_id</code>	string	Yes
<code>asset_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No

Metadata tools

These tools hit the Dataplex Metadata Service, which exposes the logical entities (tables and filesets) discovered inside a zone along with their schemas and partitions.

`list_entities`

List entities in a zone.

Arg	Type	Required	Notes
lake_id	string	Yes	
zone_id	string	Yes	
view	enum TABLES FILESETS	No	Restrict to one entity kind.
project_id	string	No	
location	string	No	
page_size	integer	No	
filter	string	No	

Returns — JSON array of entity summaries. Notable fields: `id`, `type` (TABLE / FILESET), `asset`, `data_path`, `system` (e.g. CLOUD_STORAGE, BIGQUERY), `format`.

get_entity

Fetch a single entity including its schema.

Arg	Type	Required	Notes
lake_id	string	Yes	
zone_id	string	Yes	
entity_id	string	Yes	
view	enum BASIC FULL SCHEMA	No	Defaults to FULL.
project_id	string	No	
location	string	No	

Returns — entity object with `schema.fields` (name, type, mode), `schema.partition_fields`, `compatibility` status, and pointers to the backing asset.

list_partitions

List partitions for an entity (useful for partitioned BigQuery or Hive-style filesets).

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>zone_id</code>	string	Yes
<code>entity_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No
<code>page_size</code>	integer	No
<code>filter</code>	string	No

Returns — JSON array with each partition's `values`, `location`, and `etag`.

Task tools

Dataplex tasks run scheduled or on-demand Spark or notebook jobs against lake resources.

`list_tasks`

List tasks defined on a lake.

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No
<code>page_size</code>	integer	No
<code>filter</code>	string	No

Returns — array of task objects including `trigger_spec` (schedule/on-demand), `execution_spec` (service account, args), `spark` or `notebook` configuration block, and `execution_status`.

get_task

Fetch a single task.

Arg	Type	Required
lake_id	string	Yes
task_id	string	Yes
project_id	string	No
location	string	No

run_task

Trigger an on-demand run of a task. Requires write IAM (`roles/dataplex.editor` or a narrower role with `dataplex.tasks.run`).

Arg	Type	Required
lake_id	string	Yes
task_id	string	Yes
project_id	string	No
location	string	No

Returns — the created `Job` resource, including its `name` (which can be fed back into `list_jobs` or future status-polling calls) and `state`.

list_jobs

List execution jobs for a task.

Arg	Type	Required
<code>lake_id</code>	string	Yes
<code>task_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No
<code>page_size</code>	integer	No

Returns — array of job objects with `state` (`RUNNING`, `SUCCEEDED`, `FAILED`, `CANCELLED`), `start_time`, `end_time`, and `message` for failures.

Data scan tools

Data scans are Dataplex's managed data quality (DQ) and data profile jobs. A scan is a reusable definition; each invocation produces a `DataScanJob` with results.

`list_data_scans`

List data scans in a project/location.

Arg	Type	Required
<code>project_id</code>	string	No
<code>location</code>	string	No
<code>page_size</code>	integer	No
<code>filter</code>	string	No

Returns — array of scan summaries, each with `type` (`DATA_QUALITY` or `DATA_PROFILE`), `data` (the source), and `execution_spec`.

`get_data_scan`

Fetch a single data scan.

Arg	Type	Required	Notes
<code>data_scan_id</code>	string	Yes	
<code>view</code>	enum <code>BASIC</code> <code>FULL</code>	No	<code>FULL</code> returns the rules/profile spec.
<code>project_id</code>	string	No	
<code>location</code>	string	No	

`run_data_scan`

Trigger an on-demand data scan run.

Arg	Type	Required
<code>data_scan_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No

Returns — the created `DataScanJob` resource.

`list_data_scan_jobs`

List jobs for a scan.

Arg	Type	Required
<code>data_scan_id</code>	string	Yes
<code>project_id</code>	string	No
<code>location</code>	string	No
<code>page_size</code>	integer	No

`get_data_scan_job`

Fetch a specific data scan job. Use `view=FULL` to see rule-level results for data quality scans.

Arg	Type	Required	Notes
<code>data_scan_id</code>	string	Yes	
<code>job_id</code>	string	Yes	
<code>view</code>	enum <code>BASIC</code> <code>FULL</code>	No	Defaults to <code>BASIC</code> .
<code>project_id</code>	string	No	
<code>location</code>	string	No	

Returns (FULL view) — includes `data_quality_result.rules_passing_percent`, `data_quality_result.rules[*]` with `passed`, `evaluated_count`, `passing_count`, and rule-specific metadata.

Filter expression syntax

Dataplex uses the standard [Google AIP-160](#) filter language. Examples:

Expression	Meaning
<code>labels.env=prod</code>	Resources with label <code>env=prod</code> .
<code>state=ACTIVE</code>	Only active resources.
<code>type=CURATED</code>	Curated zones only.
<code>create_time > "2024-01-01T00:00:00Z"</code>	Created after a given instant.

Response size and paging

The server does **not** transparently paginate — it issues a single `ListX` request per tool call. If you need to walk large result sets, pass an explicit `page_size` and make successive calls, or add a new tool that wraps the `ListX` iterator. See Development for the pattern.

Architecture

This page explains how the Dataplex MCP Server is structured, how a request flows through it, and the design trade-offs behind the current layout.

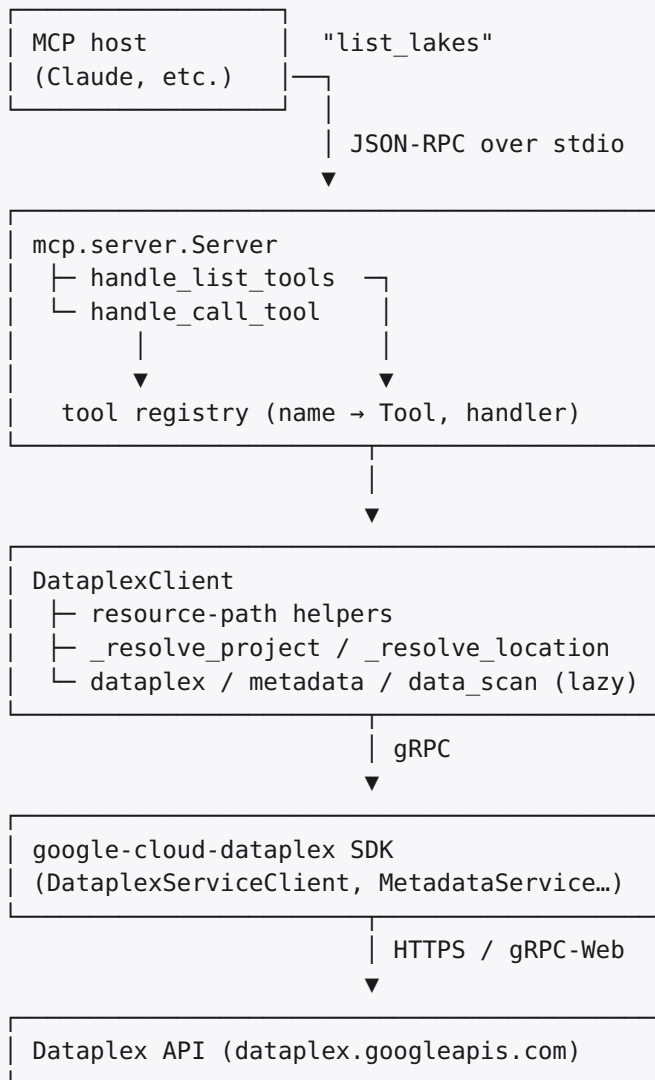
Module layout

```
dataplex-mcp-server/
├── pyproject.toml           # packaging, deps, console script
├── README.md               # short overview (points at docs/)
├── docs/                   # this documentation
├── src/dataplex_mcp_server/
│   ├── __init__.py         # package version
│   ├── __main__.py         # `python -m dataplex_mcp_server`
│   ├── dataplex_client.py  # Google Cloud SDK facade
│   └── server.py           # MCP server + tool registry
└── tests/
    └── test_dataplex_client.py # path / config / enum tests
```

Three concepts worth naming up front:

- **DataplexConfig** — a frozen dataclass that holds the default project and location. Built from explicit arguments or from the environment via `DataplexConfig.from_env()`.
- **DataplexClient** — a facade over three Google Cloud service clients (`DataplexServiceClient`, `MetadataServiceClient`, `DataScanServiceClient`). It owns resource-path construction and converts proto responses to plain dicts.
- **Tool registry** — a list of `mcp.types.Tool` objects paired with callables, built in `server._build_tool_registry`. The MCP `Server` instance uses `handle_list_tools` and `handle_call_tool` handlers to route traffic through the registry.

Request flow



Concretely, when the host calls `list_lakes`:

1. The host sends an MCP `tools/call` frame with `name="list_lakes"` and `arguments={...}`.
2. `handle_call_tool` looks up the handler by name and invokes it with the arguments as keyword arguments.
3. The handler (a lambda in the registry) calls `DataplexClient.list_lakes(...)`.
4. `DataplexClient` resolves the effective project/location, builds a `ListLakesRequest`, and dispatches it through the lazily-created `DataplexServiceClient`.
5. The SDK returns a paged iterator of proto messages; each is converted to a plain dict with `proto.Message.to_dict` (or `MessageToDict` as a fallback).

6. The resulting list is serialized to JSON via `json.dumps(..., default=str)` and wrapped in a single `TextContent` item in the MCP response.

Design decisions

Stdio transport

The server only supports stdio. Local subprocess transport is the de-facto standard for single-user MCP servers and maps cleanly onto the way MCP hosts already launch tools. HTTP/WebSocket support could be added later with `mcp.server.sse` or a custom transport layer without changing the tool registry.

Low-level `mcp.server.Server` instead of `FastMCP`

`FastMCP` generates tool schemas from Python type hints. That's ergonomic for small projects but hides what the MCP host actually sees. Using the low-level `Server` API means every tool ships with an **explicit JSON Schema**, so:

- Hosts get precise validation errors.
- Enums (`view`), numeric bounds (`page_size` 1–1000), and descriptions are first-class.
- The schema is trivially testable and diffable across versions.

Facade class instead of raw SDK calls in the server

Three benefits:

- **Testability.** Path construction and config resolution can be unit-tested without the real SDK.
- **Lazy client construction.** `DataplexClient` only builds a given service client the first time it's used, so an entirely unused `DataScanServiceClient` never gets built.
- **Consistent error surface.** All SDK calls funnel through one place, making it easy to add retries, metrics, or caching later.

Proto → dict conversion

Returning proto messages directly would require the host to understand the Dataplex protobuf schema. Converting to JSON-friendly dicts makes results self-describing and LLM-friendly, at the cost of losing strict typing. `_to_dict` handles:

1. proto-plus messages (`proto.Message`) via their class-level `to_dict`.
2. Classic `google.protobuf.Message` via `MessageToDict` (`preserving_proto_field_name=True`, so field names stay snake_case).

3. Already-dict values, passed through untouched.
4. `None`, returned as `{}`.

Error surfacing

Rather than letting exceptions propagate up and become opaque MCP protocol errors, `handle_call_tool` catches them and returns a JSON body with `error` and `message`. This gives the LLM something it can read and act on (for example, retrying with a valid argument).

Single-file tool registry

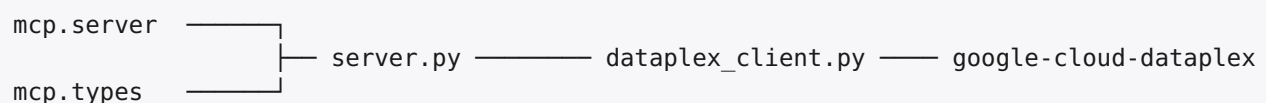
All tool registrations live in one function, `_build_tool_registry`. With 18 tools this is still easy to scan; if the server grows past ~40 tools, splitting the registry by functional area (lakes, metadata, scans, tasks) is the obvious refactor.

Extension points

Intentionally shaped so you can plug into them:

Hook	How to use it
Add a tool	Add a <code>register(...)</code> call inside <code>_build_tool_registry</code> .
Add a Dataplex operation	Add a method on <code>DataplexClient</code> , then a tool that calls it.
Inject a custom client (tests)	<code>build_server(client=my_fake)</code> bypasses the default <code>DataplexClient</code> .
Change transport	Replace <code>stdio_server()</code> in <code>_run()</code> with another MCP transport.
Add observability	Wrap <code>DataplexClient</code> methods in a logging/metrics decorator; the server layer is untouched.

Dependency boundaries



- `dataplex_client.py` depends on `google.cloud.dataplex_v1` but **not** on `mcp`.

- `server.py` depends on both — it is the only place the two worlds meet.
- Unit tests stub `google.cloud.dataplex_v1` so that `dataplex_client.py` can be exercised without network access.

This separation means the client class could be reused from a different protocol layer (e.g. a REST wrapper, a CLI, or a different MCP framing layer) with zero changes.

Development

Guide for contributors: repository layout, local setup, testing, linting, and the recipe for adding new tools.

Local setup

```
git clone https://github.com/cpydimuk/cdap.git
cd cdap/dataplex-mcp-server

python -m venv .venv
source .venv/bin/activate
pip install -e '[dev]'
```

The `[dev]` extra pulls in `pytest`, `pytest-asyncio`, and `ruff`.

Running tests

```
pytest                # run all tests
pytest -v             # verbose
pytest tests/test_dataplex_client.py::test_lake_path
```

The existing suite avoids any real Google Cloud calls. A stub for `google.cloud.dataplex_v1` is installed in `tests/test_dataplex_client.py` before the client module is imported, so the tests run offline.

If you want to run **integration** tests against a real Dataplex project, set `DATAPLEX_PROJECT_ID` / `DATAPLEX_LOCATION` and drive the `DataplexClient` methods directly. Integration tests are intentionally **not** committed to the repository — keep them out of CI unless you gate them behind a secret-bearing environment.

Linting and formatting

```
ruff check .
ruff format .
```

The project targets Python 3.10+ with these rule sets: `E`, `F`, `W`, `I`, `B`, `UP` (see `[tool.ruff.lint]` in `pyproject.toml`). Line length is 100 characters.

Adding a new tool

The server is designed so that adding a tool is a three-step change in at most two files. Worked example: let's add `cancel_job`, which cancels a running Dataplex task job.

Step 1 — add the operation to `DataplexClient`

Open `src/dataplex_mcp_server/dataplex_client.py` and add a method:

```
def cancel_job(
    self,
    lake_id: str,
    task_id: str,
    job_id: str,
    project_id: str | None = None,
    location: str | None = None,
) -> dict[str, Any]:
    name = (
        f"{self.task_path(project_id, location, lake_id, task_id)}"
        f"/jobs/{job_id}"
    )
    request = dataplex_v1.CancelJobRequest(name=name)
    # CancelJob returns google.protobuf.Empty; return a simple ack.
    self.dataplex.cancel_job(request=request)
    return {"cancelled": name}
```

Reuse existing path helpers wherever possible so that project/location resolution stays in one place.

Step 2 — register the tool in `server.py`

Inside `_build_tool_registry`, add a `register(...)` call in the Tasks section:

```

register(
    name="cancel_job",
    description="Cancel a running Dataplex task job.",
    schema=_schema(
        {
            "lake_id": {"type": "string"},
            "task_id": {"type": "string"},
            "job_id": {"type": "string"},
            **_LOCATION_PROPS,
        },
        required=["lake_id", "task_id", "job_id"],
    ),
    handler=lambda **kw: client.cancel_job(
        lake_id=kw["lake_id"],
        task_id=kw["task_id"],
        job_id=kw["job_id"],
        project_id=kw.get("project_id"),
        location=kw.get("location"),
    ),
)

```

Step 3 — add tests

Add unit tests in `tests/test_dataplex_client.py` for any new path helpers or argument parsing. You can also assert that the registry contains the new tool:

```

from dataplex_mcp_server.dataplex_client import DataplexClient, DataplexConfig
from dataplex_mcp_server.server import _build_tool_registry

def test_cancel_job_registered():
    client = DataplexClient(DataplexConfig(project_id="p", location="us-central1"))
    tools, handlers = _build_tool_registry(client)
    assert "cancel_job" in handlers
    tool = next(t for t in tools if t.name == "cancel_job")
    assert set(tool.inputSchema["required"]) == {"lake_id", "task_id", "job_id"}

```

Step 4 — document it

Add an entry under the right section of `docs/tools-reference.md`. Keep the argument table, return shape, and an example call consistent with the existing entries.

Conventions

- **Snake case everywhere.** Field names in responses use Dataplex's `snake_case`, preserved by `preserving_proto_field_name=True`.

- **Argument names match Dataplex resource segments** (`lake_id`, `zone_id`, `asset_id`, `entity_id`, `task_id`, `data_scan_id`, `job_id`). This keeps prompts and tool calls easy to reason about.
- **`filter_` in Python, `filter` in JSON schema.** Python reserves `filter`, so the Python signature uses `filter_`, but the MCP schema and user-facing argument name is `filter`.
- **Return dicts, not proto messages.** Always run results through `_to_dict` before handing them back to the server layer.
- **Fail loudly.** Validation errors should raise `ValueError` with a message that explains what was missing and how to fix it.

Release checklist

When cutting a new version:

1. Bump `version` in `pyproject.toml` and `__init__.py`.
2. Update `SERVER_VERSION` in `server.py`.
3. Update `CHANGELOG.md` (create it the first time).
4. Run the full suite: `pytest && ruff check .`
5. Tag the commit: `git tag dataplex-mcp-server-v0.2.0`.
6. Build and publish: `python -m build && twine upload dist/*` (if publishing to a package index).

Continuous integration

The project currently relies on the parent CDAP build pipeline and `pytest` / `ruff` run locally. When adding CI:

- Run tests on Python 3.10, 3.11, 3.12.
- Cache `~/.cache/pip`.
- Never set real Google Cloud credentials in public CI. Any integration tests must run in a private lane with Workload Identity Federation.

Debugging tips

- **See raw MCP traffic.** Run the host with its debug flag (e.g. Claude Desktop's developer log) and look at the JSON frames going in and out.
- **Run a handler directly.** Use the smoke-test snippet from Getting Started — it bypasses the MCP transport entirely and surfaces underlying API errors.

- **Enable SDK logging.**

```
import logging
logging.getLogger("google.api_core").setLevel(logging.DEBUG)
logging.getLogger("google.auth").setLevel(logging.DEBUG)
```

Useful for diagnosing permission and auth errors.

Troubleshooting

Common errors when running the Dataplex MCP Server and how to resolve them.

Authentication and identity

`DefaultCredentialsError: Could not automatically determine credentials`

ADC is not set up on the machine running the server. Pick one:

- Run `gcloud auth application-default login` for interactive user credentials.
- Set `GOOGLE_APPLICATION_CREDENTIALS` to a service-account key file path.
- On GCE/GKE/Cloud Run, attach a service account to the resource.

Then restart the MCP host so it re-launches the server with the new environment.

`PermissionDenied: 403 Permission 'dataplex.lakes.list' denied`

The identity the server is running as lacks Dataplex IAM roles.

```
gcloud projects add-iam-policy-binding MY_PROJECT \  
  --member="user:me@example.com" \  
  --role="roles/dataplex.viewer"
```

For service accounts:

```
gcloud projects add-iam-policy-binding MY_PROJECT \  
  --member="serviceAccount:SA_EMAIL" \  
  --role="roles/dataplex.viewer"
```

Use `roles/dataplex.editor` if you need `run_task` or `run_data_scan`.

Wrong project used

Symptom: tools succeed but list resources from an unexpected project.

Cause: `GOOGLE_CLOUD_PROJECT` is set globally and shadowing your intended project. Set `DATAPLEX_PROJECT_ID` explicitly to take precedence, or pass `project_id` in the tool call.

Configuration errors

ValueError: project_id is required

Neither `project_id` was passed to the tool nor `DATAPLEX_PROJECT_ID` / `GOOGLE_CLOUD_PROJECT` is set. Fix by setting the env var in the MCP host config:

```
"env": {
  "DATAPLEX_PROJECT_ID": "my-gcp-project",
  "DATAPLEX_LOCATION": "us-central1"
}
```

ValueError: location is required

Same cause for `DATAPLEX_LOCATION`. Dataplex is region-scoped, so a location is always required.

ValueError: Invalid entity view 'X'

The `view` argument only accepts `TABLES` or `FILESETS` (for `list_entities`) or `BASIC` / `FULL` / `SCHEMA` (for `get_entity`). Case is ignored.

API errors

NotFound: 404

The resource name doesn't exist in the target project/location. Double-check IDs and the region — a lake in `us-central1` is not visible from `eu-west-1`.

InvalidArgument: Request contains an invalid argument

Usually caused by a malformed filter expression. Start without a filter, confirm the tool works, then add the filter incrementally.

See tool reference for examples.

FailedPrecondition when running a task or scan

The task or scan exists but is in a state that can't be run — for example, a task currently disabled, or a data scan whose source asset no longer exists. Use `get_task` / `get_data_scan` to inspect state first.

Long-running requests time out

The server does not currently impose its own timeout; timeouts come from the gRPC layer. If you see `DeadlineExceeded`, the underlying API call took too long. Options:

- Narrow the scope (smaller `page_size`, tighter filter).
- Fetch a single resource with `get_*` instead of listing.
- For data scan results, use `view=BASIC` first, and only fetch `view=FULL` for the specific job of interest.

MCP host issues

The host doesn't see the tools

Check, in order:

1. The host's MCP server config points at a real, executable binary. Run that binary manually to confirm it starts. If you use a virtualenv, the path must be the venv's `bin/dataplex-mcp-server`.
2. The host was **restarted** after the config change — most hosts only read the config on startup.
3. The host's logs show the server launching. Claude Desktop logs live at `~/Library/Logs/Claude/mcp*.log` on macOS.
4. No stdout writes from a custom wrapper script. Anything printed to stdout that isn't MCP framing will corrupt the stream.

"Server exited immediately"

Usually an import error or missing dependency. Run the server manually to see the traceback:

```
dataplex-mcp-server < /dev/null
```

If it prints a Python traceback, fix the underlying issue (missing `google-cloud-dataplex`, wrong Python version, etc.) and try again.

Tools appear but every call returns an error

Check the host logs for the error payload; the server returns a JSON body like `{"error": "PermissionDenied", "message": "..."}` rather than crashing. The `error` field is the Python exception class, which maps directly to a Google API status.

Protocol and framing

JSON decode errors in the host logs

Something is writing non-MCP content to the server's stdout. Common causes:

- A `print(...)` call accidentally left in custom code. Replace it with `logger.info(...)` — the server's logging goes to stderr.
- A wrapper script that echoes commands before exec'ing the server.

Unicode issues on Windows

Force UTF-8 on the host launch by setting `PYTHONIOENCODING=utf-8` in the MCP host's env block. The server itself serializes responses with `json.dumps` which is ASCII-safe by default.

Performance

Listing a large project is slow

Dataplex pagination defaults are generous; a single `list_lakes` call in a project with hundreds of lakes can take several seconds. Use a `filter` expression to narrow the result set — for example, filtering on `labels.team=platform` or `state=ACTIVE`.

Repeated identical calls are slow

The server does not cache responses. If you need caching — for example, a model that repeatedly asks for the same schema — add a memoizing decorator around the `DataplexClient` method or implement a cache layer in the calling host.

Still stuck?

1. Reproduce the problem with the smoke-test snippet from Getting Started. That bypasses MCP entirely and tells you whether the issue is Dataplex-side or MCP-side.
2. Enable Google SDK debug logging (see Development).
3. File an issue in the parent CDAP repository with the tool call, the arguments, and the full error payload.